

Module 4: Constraints, Database Design & Normalization

Q1. For the loan table, list one example of each: a super key, a candidate key, the primary key, and a foreign key. State why your super key is not a candidate key.

A super key can be (loan_id, principal_amount). A candidate key can be loan_id. The primary key is loan_id. A foreign key is customer_id because it refers to customer.customer_id. The super key is not a candidate key because principal_amount can be removed and loan_id still uniquely identifies the loan.

Q2. ICICI Bank proposes using email as the primary key of customer. Give three reasons to refuse, and say what you would do instead.

Email can change, it is a relatively large business value to repeat in child tables, and it represents business information rather than stable database identity. I would use customer_id as the primary key and add UNIQUE(email) so email is still required to be unique.

Q3. Explain RESTRICT and CASCADE, then decide the correct rule for each relationship.

RESTRICT stops deletion of a parent when dependent rows exist. CASCADE deletes dependent rows automatically. account_txn → account: RESTRICT, because transaction history must not disappear. card_reward_enrolment → credit_card: CASCADE, because enrolment belongs to the card. loan → customer: RESTRICT, because a customer's loan must not be deleted with the customer. employee.manager_id → employee: SET NULL, because employees must remain if their manager leaves. loan_payment → loan: CASCADE, because a payment schedule has no meaning without its loan.

Q4. Define insert, update and delete anomalies and give the branch_customer_flat example.

Insert anomaly: an unrelated fact cannot be added independently; a new branch cannot be recorded cleanly without a customer. Update anomaly: the same fact must be changed in many rows; BR03 appeared with different branch-name spellings. Delete anomaly: deleting one fact destroys another; deleting an account could also remove the customer's phone because both were stored in one row.

Q5. Read the crow's foot notation and write it in words.

BRANCH ||-----o{ EMPLOYEE means every employee belongs to exactly one branch, while a branch can have zero or many employees. LOAN ||-----|{ LOAN_PAYMENT means every loan has one or more payment rows, and every payment belongs to exactly one loan.

Q6. Explain why a many-to-many relationship needs a junction table.

A foreign key on one side represents one-to-many, not many-to-many. A junction table stores one row for each relationship. Its primary key is normally the pair of foreign keys, which prevents the same pair from being stored twice.

Q7. State the rule and anomaly removed for 1NF, 2NF, 3NF and BCNF.

1NF: atomic values and no repeating groups; removes problems caused by multi-valued and repeating columns. 2NF: no partial dependency on a composite key; removes repeated parent facts caused by partial dependency. 3NF: no transitive dependency; removes update inconsistencies and duplicated lookup facts. BCNF: every determinant is a candidate key; removes remaining redundancy caused by overlapping candidate keys.

Q8. Why can a table with a single-column primary key never violate 2NF?

2NF concerns a non-key column depending on only part of a composite key. A one-column key has no smaller part, so partial dependency is impossible. Therefore a 1NF table with a single-column key is automatically in 2NF.

Q9. For each table, name the violated normal form and the fix.

1. customer(phone_1, phone_2, phone_3): 1NF violation; create customer_phone with one phone per row. 2. order_line(order_id, product_id, quantity, product_name, product_price): 2NF violation when (order_id, product_id) is the key; move product details to product. 3. employee(employee_id, name, branch_id, branch_name, branch_city): 3NF violation; move branch details to branch. 4. account(account_id, customer_id, customer_name, balance): 3NF violation; keep customer_name in customer and customer_id as the foreign key.

Q10. account.balance is derivable from account_txn and is stored anyway. Explain.

It is deliberate denormalization. The balance is stored because reading a balance should not require scanning a long transaction history. A control such as updating both values in one transaction and reconciliation must exist to detect or prevent drift.

Q11. Write queries proving branch_ifsc depends on branch_code and branch_name does not.

Group by branch_code and look for more than one distinct value. No rows from the first query means branch_code determines branch_ifsc. Rows from the second query identify inconsistent branch names.

```
SELECT branch_code
FROM branch_customer_flat
GROUP BY branch_code
```

```
HAVING COUNT(DISTINCT branch_ifsc) > 1;
```

```
SELECT branch_code, COUNT(DISTINCT branch_name) AS different_names  
FROM branch_customer_flat  
GROUP BY branch_code  
HAVING COUNT(DISTINCT branch_name) > 1;
```

Q12. Report repeated branch details and bytes used versus one stored copy.

Count the rows carrying each branch name and compare the total character length with the length of one representative name.

```
SELECT branch_code,  
       COUNT(*) AS repeated_rows,  
       SUM(LENGTH(branch_name)) AS bytes_stored,  
       MAX(LENGTH(branch_name)) AS bytes_if_stored_once  
FROM branch_customer_flat  
GROUP BY branch_code  
ORDER BY branch_code;
```

Q13. Show that cust_phone is unique in the current data and explain why it should not be the primary key.

If the query returns no rows and all phones are non-null, the current data has one row per phone. It is still unsuitable as a primary key because a customer can change a phone number.

```
SELECT cust_phone, COUNT(*) AS occurrences  
FROM branch_customer_flat  
GROUP BY cust_phone  
HAVING COUNT(*) > 1;
```

Q14. Decompose branch_customer_flat into 1NF using ex4_customer_account and ex4_customer_product.

Each repeating account becomes a separate row and each product in the comma-separated products value becomes a separate row. The COUNT queries give the resulting row counts.

```
CREATE TABLE ex4_customer_account (  
    customer_id INTEGER NOT NULL,  
    account_no VARCHAR(30) NOT NULL,  
    balance NUMERIC(18,2),  
    CONSTRAINT pk_ex4_customer_account PRIMARY KEY (customer_id, account_no)  
);
```

```
INSERT INTO ex4_customer_account(customer_id, account_no, balance)  
SELECT customer_id, account_no_1, balance_1 FROM branch_customer_flat WHERE  
account_no_1 IS NOT NULL  
UNION ALL  
SELECT customer_id, account_no_2, balance_2 FROM branch_customer_flat WHERE  
account_no_2 IS NOT NULL  
UNION ALL  
SELECT customer_id, account_no_3, balance_3 FROM branch_customer_flat WHERE  
account_no_3 IS NOT NULL;
```

```

CREATE TABLE ex4_customer_product (
    customer_id INTEGER NOT NULL,
    product VARCHAR(20) NOT NULL,
    CONSTRAINT pk_ex4_customer_product PRIMARY KEY (customer_id, product)
);

INSERT INTO ex4_customer_product(customer_id, product)
SELECT customer_id, TRIM(x.product)
FROM branch_customer_flat f
CROSS JOIN LATERAL regexp_split_to_table(f.products, ',') AS x(product)
WHERE NULLIF(TRIM(x.product), '') IS NOT NULL;

SELECT COUNT(*) FROM ex4_customer_account;
SELECT COUNT(*) FROM ex4_customer_product;

```

Q15. Decompose to 3NF into ex4_branch and ex4_customer. Handle the four BR03 spellings.

Branch details belong in one branch table and customer rows keep only the branch foreign key. For BR03, do not choose a spelling merely by frequency; verify it against the authoritative branch/IFSC source and store the verified name once.

```

CREATE TABLE ex4_branch (
    branch_id INTEGER PRIMARY KEY,
    branch_code VARCHAR(20) NOT NULL UNIQUE,
    branch_name VARCHAR(100) NOT NULL,
    branch_ifsc VARCHAR(20) NOT NULL UNIQUE,
    branch_city VARCHAR(100) NOT NULL
);

CREATE TABLE ex4_customer (
    customer_id INTEGER PRIMARY KEY,
    first_name VARCHAR(100) NOT NULL,
    last_name VARCHAR(100),
    cust_phone VARCHAR(20),
    branch_id INTEGER NOT NULL,
    CONSTRAINT fk_ex4_customer_branch
        FOREIGN KEY(branch_id) REFERENCES ex4_branch(branch_id)
        ON DELETE RESTRICT
);

```

Q16. Prove all three anomalies are gone from the decomposed design.

An insert can add a branch independently. A branch-name update changes one stored row. Deleting an account does not delete the customer because customer and account are separate entities. The foreign keys also stop unsafe parent deletion.

```

INSERT INTO
ex4_branch(branch_id,branch_code,branch_name,branch_ifsc,branch_city)
VALUES (999,'BR99','Test Branch','TEST0000001','Mumbai');

UPDATE ex4_branch
SET branch_name='Test Branch Updated'
WHERE branch_code='BR99';

```

```
-- Delete an account in the normalized account table and verify that
-- the customer row remains. Parent deletes are protected by RESTRICT.
```

Q17. Design the locker-service schema with named constraints and deliberate cascading rules.

The locker must exist even when it is unused, so locker is independent of agreement. An agreement connects a customer to a locker for a period. Visits depend on an agreement and can therefore be cascaded when the agreement is removed. Master and financial entities use RESTRICT.

```
CREATE TABLE ex4_locker (
    locker_id BIGINT GENERATED ALWAYS AS IDENTITY,
    branch_id INTEGER NOT NULL,
    locker_size VARCHAR(20) NOT NULL,
    annual_rent NUMERIC(12,2) NOT NULL,
    CONSTRAINT pk_ex4_locker PRIMARY KEY(locker_id),
    CONSTRAINT fk_ex4_locker_branch FOREIGN KEY(branch_id)
        REFERENCES branch(branch_id) ON DELETE RESTRICT,
    CONSTRAINT ck_ex4_locker_rent CHECK(annual_rent >= 0)
);

CREATE TABLE ex4_locker_agreement (
    agreement_id BIGINT GENERATED ALWAYS AS IDENTITY,
    locker_id BIGINT NOT NULL,
    customer_id INTEGER NOT NULL,
    start_date DATE NOT NULL,
    end_date DATE,
    status VARCHAR(20) NOT NULL,
    CONSTRAINT pk_ex4_locker_agreement PRIMARY KEY(agreement_id),
    CONSTRAINT fk_ex4_agreement_locker FOREIGN KEY(locker_id)
        REFERENCES ex4_locker(locker_id) ON DELETE RESTRICT,
    CONSTRAINT fk_ex4_agreement_customer FOREIGN KEY(customer_id)
        REFERENCES customer(customer_id) ON DELETE RESTRICT,
    CONSTRAINT ck_ex4_agreement_dates
        CHECK(end_date IS NULL OR end_date > start_date),
    CONSTRAINT ck_ex4_agreement_status
        CHECK(status IN('ACTIVE','CLOSED','EXPIRED'))
);

CREATE TABLE ex4_locker_visit (
    visit_id BIGINT GENERATED ALWAYS AS IDENTITY,
    agreement_id BIGINT NOT NULL,
    visit_date DATE NOT NULL,
    visit_time TIME NOT NULL,
    employee_id INTEGER NOT NULL,
    CONSTRAINT pk_ex4_locker_visit PRIMARY KEY(visit_id),
    CONSTRAINT fk_ex4_visit_agreement FOREIGN KEY(agreement_id)
        REFERENCES ex4_locker_agreement(agreement_id) ON DELETE CASCADE,
    CONSTRAINT fk_ex4_visit_employee FOREIGN KEY(employee_id)
        REFERENCES employee(employee_id) ON DELETE RESTRICT
);
```

Q18. For the locker design, identify the weak entity, many-to-many relationship, nullable foreign key and two CHECK rules.

Locker visit is dependent on a locker agreement. Customer and locker form a many-to-many relationship over time, resolved by locker_agreement. No foreign key is nullable in this design; end_date is nullable because an active agreement may not have an end date. The CHECK rules are annual_rent >= 0 and end_date > start_date when an end date exists.

Q19. Add a CHECK constraint preventing an end date on or before the start date and write the INSERT that proves it.

The constraint rejects an agreement whose end date is earlier than or equal to its start date.

```
ALTER TABLE ex4_locker_agreement
ADD CONSTRAINT ck_ex4_agreement_dates
CHECK(end_date IS NULL OR end_date > start_date);

INSERT INTO
ex4_locker_agreement(locker_id,customer_id,start_date,end_date,status)
VALUES(1,1001,DATE '2026-01-10',DATE '2026-01-09','ACTIVE');
```

Q20. Design ex4_branch_daily_summary and populate it for January 2026.

The natural primary key is (branch_id, summary_date), because there should be one summary per branch per day.

```
CREATE TABLE ex4_branch_daily_summary (
    branch_id INTEGER NOT NULL,
    summary_date DATE NOT NULL,
    transaction_count INTEGER NOT NULL,
    total_credits NUMERIC(18,2) NOT NULL,
    total_debits NUMERIC(18,2) NOT NULL,
    closing_deposit_total NUMERIC(18,2) NOT NULL,
    CONSTRAINT pk_ex4_branch_daily_summary PRIMARY KEY(branch_id,summary_date),
    CONSTRAINT fk_ex4_summary_branch FOREIGN KEY(branch_id)
        REFERENCES branch(branch_id) ON DELETE RESTRICT
);

INSERT INTO ex4_branch_daily_summary
(branch_id,summary_date,transaction_count,total_credits,total_debits,closing_de
posit_total)
SELECT a.branch_id,t.txn_date::date,COUNT(*),
    SUM(CASE WHEN t.txn_type='CREDIT' THEN t.amount ELSE 0 END),
    SUM(CASE WHEN t.txn_type='DEBIT' THEN t.amount ELSE 0 END),
    SUM(a.balance)
FROM account a
JOIN account_txn t ON t.account_id=a.account_id
WHERE t.txn_date >= DATE '2026-01-01'
    AND t.txn_date < DATE '2026-02-01'
GROUP BY a.branch_id,t.txn_date::date;
```

Q21. Write the reconciliation query and explain a non-empty result.

A non-empty result means at least one stored summary value differs from the transactional calculation, so the denormalized summary needs investigation.

```
SELECT s.branch_id,s.summary_date,
       s.transaction_count,x.transaction_count AS calculated_count,
       s.total_credits,x.total_credits AS calculated_credits,
       s.total_debits,x.total_debits AS calculated_debits
FROM ex4_branch_daily_summary s
JOIN (
    SELECT a.branch_id,t.txn_date::date AS summary_date,
           COUNT(*) AS transaction_count,
           SUM(CASE WHEN t.txn_type='CREDIT' THEN t.amount ELSE 0 END) AS
total_credits,
           SUM(CASE WHEN t.txn_type='DEBIT' THEN t.amount ELSE 0 END) AS
total_debits
    FROM account a JOIN account_txn t ON t.account_id=a.account_id
    GROUP BY a.branch_id,t.txn_date::date
) x ON x.branch_id=s.branch_id AND x.summary_date=s.summary_date
WHERE s.transaction_count<>x.transaction_count
    OR s.total_credits<>x.total_credits
    OR s.total_debits<>x.total_debits;
```

Q22. A vendor uses ON DELETE CASCADE on every foreign key. Respond to the vendor and data governance.

Using CASCADE everywhere is unsafe for banking data. Deleting a customer could delete accounts and then transaction history. I would require RESTRICT for customer→account and account→account_txn, and allow CASCADE only where the child is genuinely owned by the parent and has no independent meaning, such as loan→loan_payment.

Q23. An architect proposes phone_1, phone_2, phone_3. Give two arguments against and one situation where it is reasonable.

It creates a repeating group and makes searching, validating and extending the data harder. It also creates an artificial limit of three. It can be reasonable when the requirement guarantees a fixed small number of phone slots and they are genuinely separate fixed attributes.

Q24. Why is branch_name repeated in a reporting warehouse acceptable but not in customer? Name the control.

A warehouse is designed for fast reporting and can deliberately duplicate descriptive data to reduce joins. Customer is an OLTP table where duplicated branch details create update inconsistencies. Controlled ETL plus reconciliation and data-quality checks keeps the warehouse copy trustworthy.

Q25. The team lead wants to choose the most frequent spelling automatically. Respond.

I would not automatically select the most frequent spelling because frequency does not prove correctness. The surviving name should be checked against an authoritative branch/IFSC source.

Automatic cleanup is appropriate for formatting such as spaces and case, but the rule and validation should be recorded.

Q26. A colleague says normalization is an academic exercise because joins are cheap and storage is free. Respond.

The main cost normalization prevents is inconsistent business information, not storage usage. Without it, one fact can exist in many places and updates can leave conflicting values, wrong reports and difficult corrections. A cheap join is usually less costly than incorrect customer or financial data.